Дискретный анализ

Вайб-лекция 4: цифровой поиск

Андрей Калинин

Дискретный анализ, 2026/27

7 октября 2026

Материалы лекций

Материалы курса — в открытом репозитории:

`https://github.com/alk-2000/da-lectures`

Здесь же регистрируемся на конкурс и задаём вопросы в Issues.



Конкурс по сортировке: регистрация открыта

Задача: самая быстрая сортировка 32-битных беззнаковых чисел. Точные условия и проверяющая программа — 14 октября. Переносим на неделю.

Срок: 6 декабря 2026, 23:59 по Москве.

Призы:

- первое место — «отлично» на экзамене;
- ещё четыре места — +1 балл на экзамене.



папка регистрации участников

После срока — семинар: победители рассказывают свои решения. Не можете объяснить своё решение — приза нет.

Как записаться на конкурс

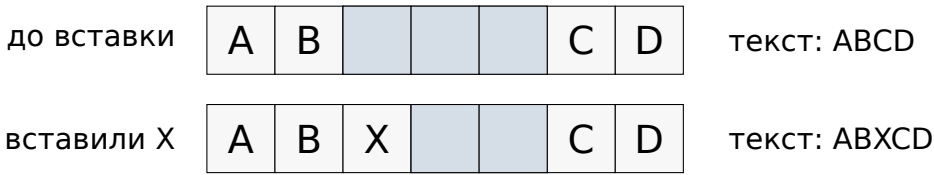
1. Откройте папку участников — QR-код на прошлом слайде, или в репозитории `alk-2000/da-lectures`: `2026/contest/sorting/participants`.
2. **Add file** → **Create new file**, имя — `<ваш ник>.md`; в файле одна строка — ваш GitHub-аккаунт, например `@octocat`.
3. **Propose new file**, затем **Create pull request**.
4. Напишите на **andrey@kalinin.ru** с темой **«Конкурс по сортировке»**: ФИО, группа, GitHub-аккаунт. В таблице результатов будут только ники.

Не выкладывайте решение в fork этого репозитория и не присылайте его pull request'ом до срока — это видят все. Для решения каждому дадим закрытый репозиторий.

Резервный буфер Emacs

Gap buffer хранит текст и ускоряет правки рядом с зазором.

Перевод offset в строку и колонку требует отдельного подсчёта переводов строк или индекса.



Синие ячейки — свободное место, **gap**. Вставка рядом с ним не сдвигает хвост текста. Для вставки в другом месте сначала перемещаем gap, копируя символы.

GNU Emacs: buffer.h, insdel.c; разбор в text-editors.md.

Вставка в случайную позицию

Начальный текст	Вставок	Gap, нс/вставку	Rope, нс/вставку
256 КиБ	20 000	544	694
2 МиБ	10 000	5 307	989
8 МиБ	3 000	31 851	1 113

Наши реализации gap buffer и rope. Каждая вставка добавляет 16 байт в новую случайную позицию; текст растёт.

Среднее время вставки в лучшем из 5 прогонов.

D3: bench-gap-vs-rope, insert-random; AMD Zen 5, C++ -O2.

14 июля 2022: исчез UNIQUE INDEX

Andreas Wolf, Python, DuckDB 0.4.0. Индекс из его примера:

```
CREATE UNIQUE INDEX customer_year_month_idx
ON raw(customer_ID, year, month);
```

- После закрытия и повторного открытия базы индекс исчезал; проверка уникальности пропадала.
- На поиск причины у пользователя ушло больше дня.

Это историческая ошибка: исправление уже было в ветке разработки.

DuckDB issue #4126, 14–15.07.2022; Andreas Wolf.

Почему открытие базы стало проблемой?

- DuckDB работает внутри приложения. При завершении процесса индекс в памяти теряется.
- В версии 0.4.0 индекс PRIMARY KEY строился заново при открытии: нужно прочитать всю таблицу.
- Даже ради одного запроса пользователь сначала платил за весь индекс.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022

Что должен уметь индекс?

code	row_id
car	17
cart	42
cat	8
dog	31

- Найти `car` и вернуть строку 17.
- Отклонить повторную вставку `car`: ключ должен быть уникален.
- Найти ключи в диапазоне: например, от `car` до `cas`, не включая `cas`.

Это наш пример на весь день. `row_id` — условный номер строки таблицы.

Почему команда выбрала один ART?

- Можно разделить работу: хеш-таблица для уникальности, B+ дерево для диапазонов.
- DuckDB выбрал ART: одна структура обслуживает обе задачи, меньше кода для сопровождения.
- Компромисс: длинные диапазоны удобнее обходить по связанным листьям B+ дерева.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022

Как мы придём к сохранённому индексу

- Сначала найдём ключ по его символам: trie.
- Затем сократим память: сжатые пути, PATRICIA и ART.
- В конце сохраним дерево и прочитаем из файла путь к `car`.

Поиск по префиксу — частный случай диапазона

- В нашем индексе начало ca имеют `car`, `cart`, `cat`.
- В отсортированном порядке они стоят подряд.
- В учебном коде будем считать такие ключи; в базе по ним получаем строки.

Проверяем начало каждой строки

```
10 inline bool starts_with(std::string_view s, std::string_view p) {
11     return s.size() >= p.size() && s.substr(0, p.size()) == p;
12 }
13
14 inline size_t scan_prefix(const std::vector<std::string>& words,
15                           std::string_view prefix) {
16     size_t count = 0;
17     for (const auto& word : words)
18         count += starts_with(word, prefix);
19     return count;
20 }
```

`starts_with(s, p)`: начинается ли строка `s` с префикса `p`?

Для `cart`, `ca` результат true; для `dog`, `ca` — false.

Учебная реализация: dictionary.hpp

В отсортированном массиве найдём начало диапазона

```
22 inline size_t sorted_prefix(const std::vector<std::string>& words,
23                             const std::string& prefix) {
24     auto it = std::lower_bound(words.begin(), words.end(), prefix);
25     size_t count = 0;
26     while (it != words.end() && starts_with(*it, prefix)) {
27         ++count;
28         ++it;
29     }
30     return count;
31 }
```

`lower_bound` находит первый ключ, не меньший `prefix`.

Дальше считаем совпадения, пока общий префикс не закончится.

Учебная реализация: dictionary.hpp

Условия нашего измерения

- 50 000 английских слов играют роль строковых ключей.
- Проверяем 512 существующих ключей и 512 отсутствующих; каждый набор повторяем 7 раз.
- Показываем медиану времени одного запроса. Построение индекса в неё не входит.

E1/E2: demo/results.csv; GCC 13.3 -O2, одно ядро AMD Zen 5.

Точный поиск: есть ли такой ключ целиком?

Метод	Ключ есть, нс	Ключа нет, нс
Перебор	15282.7	21030.1
Массив + бинарный поиск	96.1	90.7
Хеш-таблица	9.5	9.9

Хеш-таблица здесь — `std::unordered_set<string>`.

Массив использует `binary_search`: тоже точное совпадение.

Для префикса: в массиве `lower_bound` + обход; в хеш-таблице — просмотр всех ключей.

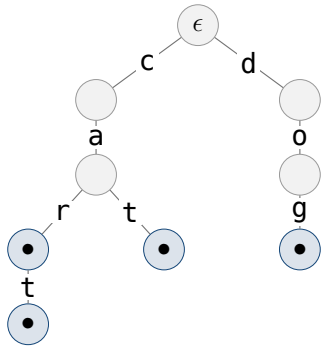
E1: наш стенд; медиана 7 проходов.

Определение: дерево ключей, или trie

- Каждое ребро помечено символом. У рёбер из одного узла метки различны.
- Символы на пути от корня образуют префикс ключа.
- Отметка в узле означает: здесь заканчивается сохранённый ключ.

Fredkin, Trie Memory (1960); Калинин, Цифровой поиск (2010).

Наши четыре ключа в trie



Точка — конец ключа. car→17, cart→42, cat→8, dog→31.

Один узел: 26 переходов и отметка конца

```
34 struct Trie {
35     struct Node {
36         std::array<int, 26> child;
37         bool terminal = false;
38         Node() { child.fill(-1); }
39     };
40     std::vector<Node> nodes{1};
```

`child[0]` соответствует a, `child[25]` — z.

-1 — ребёнка нет; иначе это индекс в `nodes`.

Код хранит только наличие ключа; в индексе добавится `row_id`.

Учебная реализация: dictionary.hpp

Поиск: пройти путь и проверить конец ключа

```
56      int walk(std::string_view key) const {
57          int v = 0;
58          for (char c : key) {
59              if (c < 'a' || c > 'z') return -1;
60              v = nodes[v].child[c - 'a'];
61              if (v == -1) return -1;
62          }
63          return v;
64      }
65      bool find(std::string_view key) const {
66          int v = walk(key);
67          return v != -1 && nodes[v].terminal;
68      }
```

`find("car")` = true, `find("ca")` = false.

Отсутствующее ребро тоже означает, что ключа нет.

Учебная реализация: dictionary.hpp

Вставка добавляет только недостающие узлы

```
42      void insert(std::string_view key) {
43          int v = 0;
44          for (char c : key) {
45              int next = nodes[v].child[c - 'a'];
46              if (next == -1) {
47                  next = static_cast<int>(nodes.size());
48                  nodes[v].child[c - 'a'] = next;
49                  nodes.emplace_back();
50              }
51              v = next;
52          }
53          nodes[v].terminal = true;
54      }
```

`nodes.emplace_back()` добавляет новый `Node` в конец вектора: 26 переходов равны -1, `terminal=false`.

Для cab добавляем к пути ca только ребро b.

Учебная реализация: dictionary.hpp

Удаление car не должно удалить cart

- В узле car снимаем отметку конца ключа.
- Его ребёнок t остаётся: он нужен для cart.
- Узел можно убрать, только если в нём не кончается ключ и нет детей.

Очистка ненужных узлов после удаления

```
77      bool prune(int v, std::string_view key, size_t pos) {
78          if (pos < key.size()) {
79              int c = key[pos] - 'a';
80              if (prune(nodes[v].child[c], key, pos + 1))
81                  nodes[v].child[c] = -1;
82          }
83          return !nodes[v].terminal &&
84              std::all_of(nodes[v].child.begin(), nodes[v].child.end(),
85                          [](int x) { return x == -1; });
```

Возвращаем true, если у узла нет ни ключа, ни детей.

Родитель в таком случае удаляет переход к нему.

Учебная реализация: dictionary.hpp

Префикс ca: найти узел и обойти его поддерево

```
87      size_t count(int v) const {
88          size_t n = nodes[v].terminal;
89          for (int c : nodes[v].child) if (c != -1) n += count(c);
90          return n;
91      }
92      size_t prefix(std::string_view p) const {
93          int v = walk(p);
94          return v == -1 ? 0 : count(v);
95      }
```

В поддереве ca три отметки: car, cart, cat.

Учебная реализация: dictionary.hpp

Сколько работы выполняет trie?

- Точный поиск ключа длины k: не более k переходов.
- При массиве детей каждый переход занимает $O(1)$, весь поиск — $O(k)$.
- Запрос по префиксу дополнительно обходит подходящее поддерево.

Сколько узлов нужно для словаря?

Что считаем	Количество
Различных слов	370 105
Символов во всех словах	3 494 700
Узлов обычного trie, включая корень	1 027 810

Это полный английский словарь. Общие начала слов уже делят путь.

В следующем шаге посмотрим, можно ли убрать ещё часть узлов.

D1: demo/stats.json; words_alpha.txt, SHA-256 в numbers.md.

Пустые переходы тоже занимают память

- У каждого нашего узла 26 индексов по 4 байта: 104 байта только на детей.
- Вместе с terminal и выравниванием `sizeof(Node)` = 108 байт.
- У узла с одним ребёнком 25 индексов не используются.

L1: GCC 13.3, x86-64; demo/results.csv.

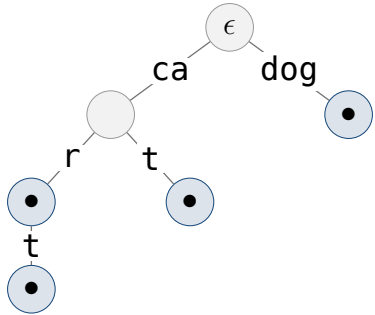
Цепочку d — o — g можно заменить одним ребром

- В промежуточных узлах нет ни других ветвей, ни концов ключей.
- Запишем на ребре сразу dog.
- Проверять символы всё ещё нужно; переходов между узлами станет меньше.

Определение: сжатое дерево ключей

- Ребро хранит строку из одного или нескольких символов.
- Первые символы рёбер из одного узла различны.
- Убираем узлы с одним ребёнком, если в них не заканчивается ключ.

Тот же индекс после сжатия путей



Конец ключа `car` сохранён.

Для полного словаря: 1 027 810 узлов trie $\rightarrow$ 478 058 узлов сжатого trie.

D1: demo/stats.json; полный словарь.

Как изменился Node после сжатия

```
105 struct Compact {
106     struct Node {
107         std::string label;
108         bool terminal = false;
109         std::map<char, std::unique_ptr<Node>> child;
110         explicit Node(std::string s = "", bool t = false)
111             : label(std::move(s)), terminal(t) {}
112     };
113     Node root;
```

`label` хранит всё входящее ребро; у корня оно пустое.

Вместо 26 ячеек — `map`: первый символ ребра → ребёнок.

`terminal` по-прежнему отмечает конец ключа.

Учебная реализация: dictionary.hpp

Сжатое ребро нужно сравнить целиком

```
132     bool find(std::string_view key) const {
133         const Node* v = &root;
134         while (!key.empty()) {
135             auto it = v->child.find(key.front());
136             if (it == v->child.end()) return false;
137             const auto& edge = *it->second;
138             if (!starts_with(key, edge.label)) return false;
139             key.remove_prefix(edge.label.size());
140             v = &edge;
141         }
142         return v->terminal;
143     }
```

В `Node`: `label` — метка ребра, `child` — map детей по первому символу.

`remove_prefix` сдвигает начало оставшейся части запроса.

Учебная реализация: dictionary.hpp

Ключ do отсутствует, но префикс do подходит

- Из корня идёт ребро dog.
- Точный запрос do закончился внутри ребра: ключ не найден.
- Префиксный запрос do совпал целиком: под этим ребром подходит всё.

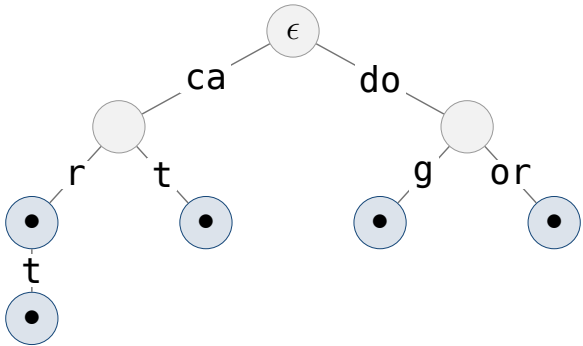
Поиск префикса внутри сжатого ребра

```
149     size_t prefix(std::string_view p) const {
150         const Node* v = &root;
151         while (!p.empty()) {
152             auto it = v->child.find(p.front());
153             if (it == v->child.end()) return 0;
154             const Node& edge = *it->second;
155             size_t k = common_prefix(edge.label, p);
156             if (k == p.size()) return count(edge);
157             if (k < edge.label.size()) return 0;
158             p.remove_prefix(k);
159             v = &edge;
160         }
161         return count(*v);
162     }
```

`common_prefix` — длина общего начала двух строк.

Учебная реализация: dictionary.hpp

Вставка door: разбиваем ребро dog



Общий участок do; после него старый хвост g и новый or.

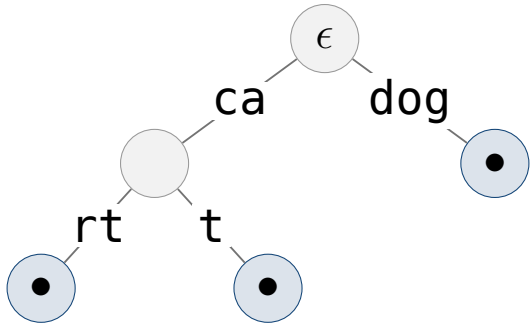
Вставка: отделяем общий префикс от старого хвоста

```
122         size_t k = common_prefix(edge->label, key);
123         if (k < edge->label.size()) {
124             auto old = std::move(edge);
125             edge = std::make_unique<Node>(old->label.substr(0, k));
126             old->label.erase(0, k);
127             char first = old->label.front();
128             edge->child[first] = std::move(old);
129         }
130         insert_at(*edge, key.substr(k));
```

k — число совпавших символов. Для dog/door: $k = 2$.

Учебная реализация: dictionary.hpp

Удаление car: соединяем r и t



Из исходных четырёх ключей удаляем car.

Остаются cart, cat, dog.

Удаление: убрать пустой узел или соединить рёбра

```
170         auto it = v.child.find(key.front());
171         if (it == v.child.end()) return false;
172         Node& e = *it->second;
173         if (!starts_with(key, e.label)) return false;
174         if (!erase_at(e, key.substr(e.label.size()))) return false;
175         if (!e.terminal && e.child.empty()) v.child.erase(it);
176         else if (!e.terminal && e.child.size() == 1) {
177             auto only = std::move(e.child.begin()->second);
178             only->label = e.label + only->label;
179             it->second = std::move(only);
180         }
181         return true;
```

Соединяем рёбра только в узле без собственного ключа и с одним ребёнком.

Учебная реализация: dictionary.hpp

Ещё один способ уменьшить статический словарь

- Trie объединяет одинаковые начала ключей.
- Если совпадают также множества продолжений, можно объединить поддеревья.
- Получится граф — DAFSA. Такой формат используется для Public Suffix List в Chromium.

Chromium, make_dafsa.py, commit 1505e40ded14.

LLM: повторные вопросы к одному документу

Два запроса к одной модели:

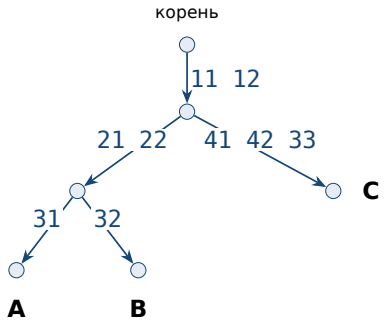
Инструкции Документ Вопрос 1
Инструкции Документ Вопрос 2

- Текст подаётся как ID токенов — номера в словаре модели.
- Перед ответом модель обрабатывает вход: **prefill**. Для следующих токенов сохраняются данные внимания — **KV-кэш**.
- Готовый KV-кэш совпадающего префикса используем снова. Новый хвост входа и ответ ещё нужно вычислить.

SGLang: RadixAttention, 2024; vLLM: Automatic Prefix Caching.

RadixAttention: три запроса в одном дереве

После A одно ребро: 11 12 21 22 31. Затем поступили B и C:



Запрос	Токенов из кэша
A	0 из 5
B	4 из 5
C	2 из 5

B разрезает ребро после 4 токенов; C — после 2.

Числа — условные ID токенов. Дерево — в памяти CPU; KV токенов — в памяти GPU.

SGLang v0.4.1; схема и числа из demo/llm_prefix.py.

SGLang: разрезаем ребро и позиции KV

```
248     def _split_node(self, key, child: TreeNode, split_len: int):
249         # new_node -> child
250         new_node = TreeNode()
251         new_node.children = {key[split_len]: child}
252         new_node.parent = child.parent
253         new_node.lock_ref = child.lock_ref
254         new_node.key = child.key[:split_len]
255         new_node.value = child.value[:split_len]
256         child.parent = new_node
257         child.key = child.key[split_len:]
258         child.value = child.value[split_len:]
259         new_node.parent.children[key[0]] = new_node
260         return new_node
```

key — токены ребра; value — позиции их KV.
split_len=4: отделяем 11 12 21 22 от 31.
lock_ref — число активных запросов, использующих узел.

SGLang v0.4.1, radix_cache.py, строки 248–260.

Памяти GPU не хватает: что удалить и кого запустить?

- Кэш и активные запросы используют один пул памяти GPU.
- Удаляем давно неиспользованный лист с `lock_ref=0`. Общий путь остаётся, пока у него есть другие дети.
- Запрос с длинным готовым префиксом требует меньше новой работы. Но его постоянный приоритет может заставить остальные запросы ждать.

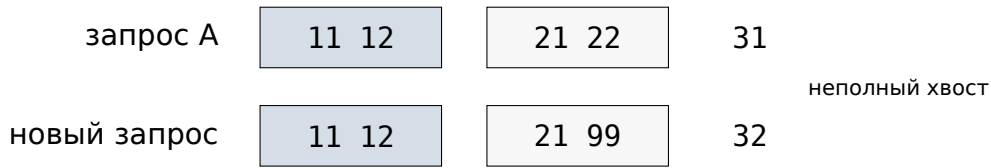
Zheng et al., SGLang, 2024, §3; SGLang v0.4.1, evict.

vLLM: кэш полных блоков токенов

V1: хеш блока учитывает префикс перед ним.

$$h_i = H(h_{i-1},\ \text{токены блока},\ \text{контекст})$$

В нашем примере блок содержит 2 токена:



Совпало 3 токена. Из кэша берём 1 блок: 2 токена.
Контекст: например, хеш входного изображения.

vLLM V1 v0.7.2: Automatic Prefix Caching; блоки на рисунке — модель.

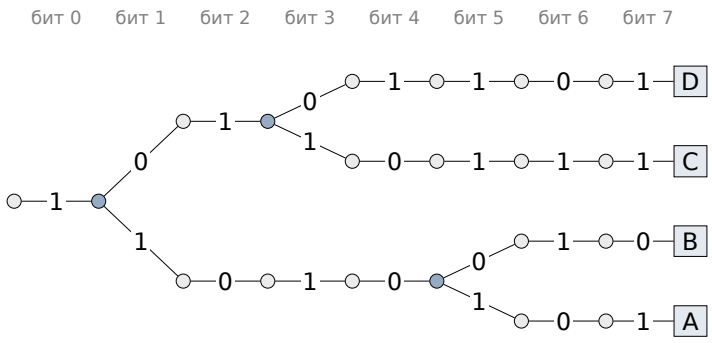
PATRICIA: разберём четыре коротких ключа

Метка	Ключ: 8 бит	Число
A	11010101	213
B	11010010	210
C	10110111	183
D	10101101	173

A–D — подписи для рисунков. Биты нумеруем слева направо от 0 до 7.

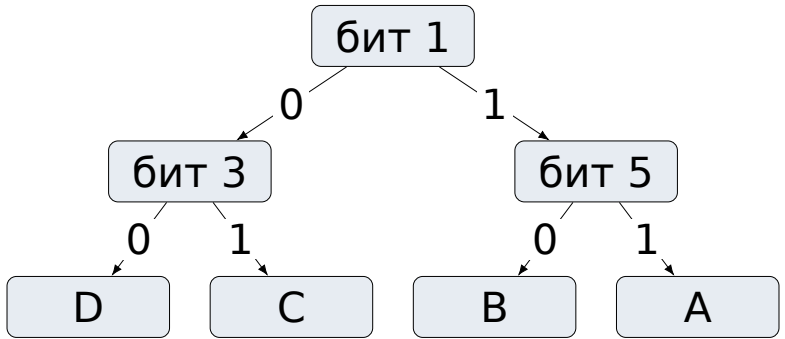
Калинин (2010); Макаров, «Цифровой поиск», слайды 8–12.

В двоичном trie много узлов с одним ребёнком



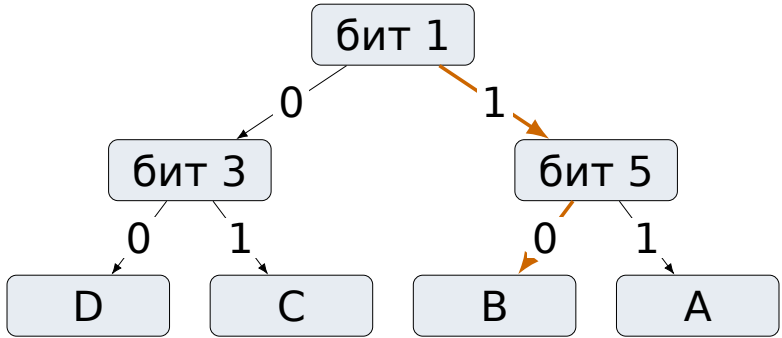
Синим выделены развилки. Только в них выбор 0 или 1 разделяет наши ключи.

Оставим развилки и номера проверяемых битов



Из корня читаем бит 1. Затем — бит 3 слева или бит 5 справа.

Ищем B: два бита, затем весь ключ



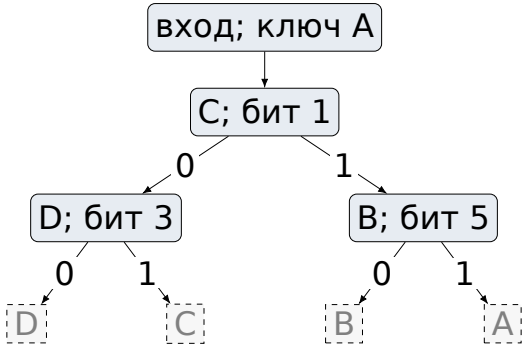
`11010010`: бит 1 = 1; бит 5 = 0.
Кандидат B совпадает с запросом.

Тот же путь может привести к отсутствующему ключу

	Восемь бит	Бит 1	Бит 5
Запрос	11010011	1	0
Кандидат B	11010010	1	0

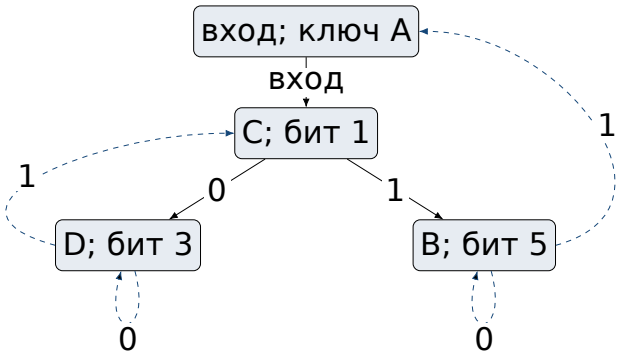
Спуск снова приведёт к B. Последний бит различается: ключа в дереве нет.

Можно хранить ключи вместе с развилками



Четыре ключа размещаем в трёх развилках и входном узле.
Серые листья пока показывают, где должен заканчиваться поиск.

Заменим каждый лист ссылкой на его ключ



Сплошная стрелка ведёт к следующей развилке.
Пунктир ведёт к ключу-кандидату и завершает поиск.

Как программа отличает конец поиска?

- В развилке храним ключ, номер бита `index` и две ссылки.
- При переходе к следующей развилке номер бита строго растёт.
- Если номер не вырос, мы пришли к кандидату. Сравниваем его ключ.
- У входного узла `index=-1`; `child[0]` ведёт в дерево.

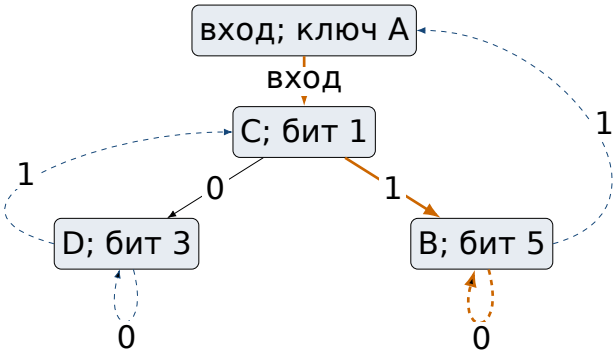
Узел и пустая PATRICIA

```
15 @dataclass(eq=False)
16 class Node:
17     key: int | None
18     index: int
19     child: list = field(default_factory=lambda: [None, None])
20
21
22 class Patricia:
23     def __init__(self):
24         self.head = Node(None, -1)
25         self.head.child = [self.head, self.head]
```

Входной узел хранит первый ключ; пустое дерево имеет key=None.

Учебная реализация: patricia8.py

Поиск B заканчивается ссылкой B на себя



Номера развилок: $1 \to 5 \to 5$.
Последний номер не вырос: сравниваем ключ B с запросом.

Поиск: спуск к кандидату и проверка равенства

```
27      def candidate(self, key):
28          parent, node = self.head, self.head.child[0]
29          while parent.index < node.index:
30              parent, node = node, node.child[bit(key, node.index)]
31          return node
32
33      def find(self, key):
34          return self.candidate(key).key == key
```

candidate находит место хранения ключа; find проверяет весь ключ.

Учебная реализация: patricia8.py

Вставляем E: где нужен новый выбор?

	Ключ
Новый E	11010111
Кандидат A	11010101

Первое различие — бит 6: у A стоит 0, у E — 1.
Новая развилка нужна после бита 5.

Вставка: найдём первый различающийся бит

```
36      def insert(self, key):
37          if self.head.key is None:
38              self.head.key = key
39              return True
40          old = self.candidate(key)
41          if old.key == key:
42              return False
43          index = 8 - (old.key ^ key).bit_length()
```

`old.key` XOR `key`: первая единица указывает на первое различие.

Учебная реализация: patricia8.py

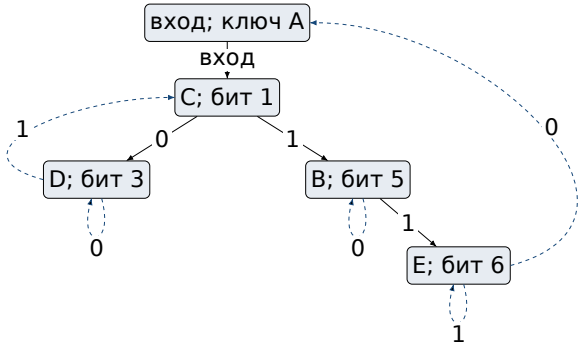
Вставка: разорвём одну ссылку

```
44          parent, node = self.head, self.head.child[0]
45          while parent.index < node.index < index:
46              parent, node = node, node.child[bit(key, node.index)]
47          new = Node(key, index)
48          side = bit(key, index)
49          new.child[side], new.child[1 - side] = new, node
50          parent.child[bit(key, parent.index)] = new
51          return True
```

Одна ссылка нового узла ведёт на него самого; другая продолжает прежний путь.

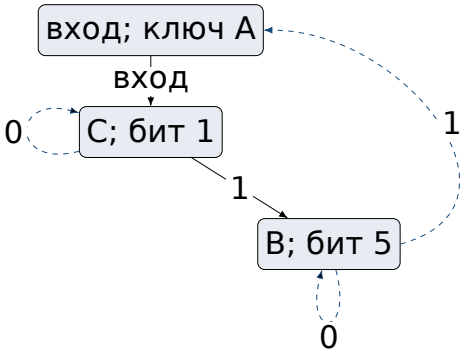
Учебная реализация: patricia8.py

После вставки E появился один узел



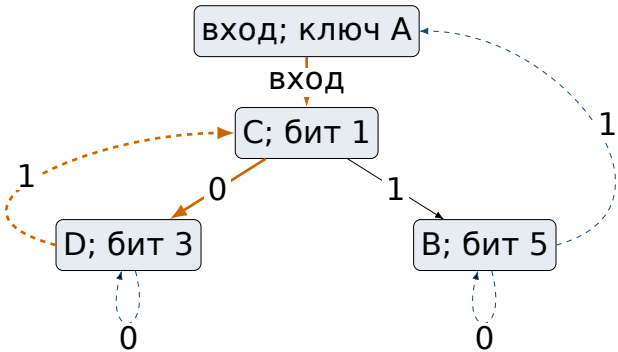
В новой развилке бит 6 = 0 завершает поиск на A; бит 6 = 1 — на E.

Удаляем D: убираем его развилку



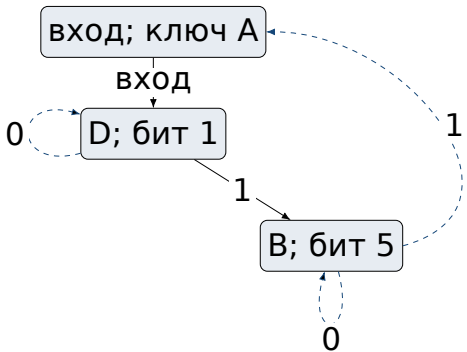
Удаление из исходного набора A–D.
Вместо перехода C → D → C оставляем завершающую ссылку C → C.

Удаляем C: его узел нужен другим поискам



Снова исходный набор A–D. Поиск C завершился переходом D → C. Узел C остаётся развилкой между левой и правой частями дерева.

Переносим ключ D, затем убираем лишнюю развилку



В узле с битом 1 теперь хранится D. Развилка с битом 3 удалена. Остались A, B и D; запрос C отвергается сравнением с D.

Удаление: найдём ключ и последнюю развилку

```
53      def erase(self, key):
54          grand = parent = self.head
55          node = self.head.child[0]
56          while parent.index < node.index:
57              grand, parent = parent, node
58              node = node.child[bit(key, node.index)]
59          if node.key != key:
60              return False
61          if parent is self.head:
62              self.head.key = None
63              return True
```

node хранит ключ-кандидат; parent — последняя развилка; grand ведёт к ней.

Учебная реализация: patricia8.py

Удаление: перенесём ключ и переподключим связь

```
64          if node is not parent:
65              # Retarget the terminating link before bypassing parent.
66              source, cur = self.head, self.head.child[0]
67              while source.index < cur.index:
68                  source, cur = cur, cur.child[bit(parent.key, cur.index)]
69              source.child[bit(parent.key, source.index)] = node
70              node.key = parent.key
71          other = parent.child[1 - bit(key, parent.index)]
72          grand.child[bit(key, grand.index)] = other
73          return True
```

Если `node == parent`, перенос не нужен. Последние три строки обходят удаляемую развилку.

Учебная реализация: patricia8.py

Для строк нужно различать конец ключа

- У `car` и `cart` общие первые три байта, но это разные ключи.
- Для строчных ASCII можно добавить байт `00`: внутри таких ключей его нет.
- Для произвольных байтов нужна кодировка, в которой конец отличается от продолжения.

Кодировка для произвольных байтов

```
10 def bit(key, index):
11     if index < 0 or key is None:
12         return 0                        # sentinel reads as zeros
13     if index == 0:
14         return 1                        # distinguish real keys
15     byte, offset = divmod(index - 1, 9)
16     if byte >= len(key):
17         return 0                        # end marker; zero padding
18     if offset == 0:
19         return 1                        # another byte follows
20     return (key[byte] >> (8 - offset)) & 1
```

Каждый байт превращается в 1 и 8 бит данных. После последнего байта читаем 0.

Учебная реализация: patricia_threaded.py

Три дерева на тех же 50 000 английских словах

Реализация	Ключ есть	Ключа нет	Префикс
Trie (26 переходов)	17.1	19.4	11180.9
Сжатый trie	89.1	83.1	1204.2
PATRICIA	99.6	103.4	603.2

Наносекунды на запрос; префикс — **подсчёт всех совпадений**.

Все три реализации — C++17, -02, одно ядро.

Здесь trie быстрее при точном поиске; PATRICIA — при подсчёте по префиксу.

E1/E2: demo/bench.cpp, dictionary.hpp, patricia.hpp.

Что выбирать: trie, сжатый trie или PATRICIA?

- Trie с массивом: простой переход по символу; цена — пустые ячейки и длинные цепочки.
- Сжатый trie: меньше узлов на длинных общих путях; всё ещё сравниваем метки рёбер.
- PATRICIA: два перехода на развилку и пропуск одинаковых битов; храним и сравниваем полный ключ.

Вернёмся к индексу DuckDB: бит или байт за шаг?

- PATRICIA выбирает ветвь по одному различающемуся биту.
- ART выбирает ветвь по целому байту: возможны 256 значений.
- Как хранить развилку экономно, если у неё 2, 20 или 200 детей?

Определение: адаптивное radix-дерево (ART)

- Ключ кодируется последовательностью байтов; цепочки общих байтов сжаты.
- Внутренний узел хранит общий префикс и переходы по следующему байту.
- Форма узла зависит от числа детей. Лист хранит оставшийся хвост ключа и номера строк.

Leis, Kemper, Neumann. ART, ICDE 2013; DuckDB v0.5.0, commit 109f932c41fb

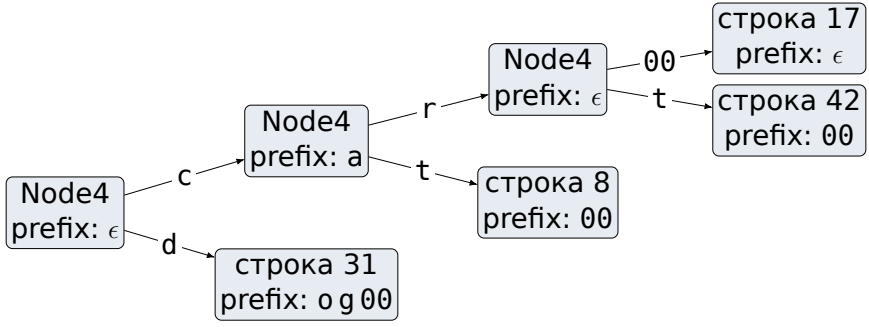
Конец строки тоже участвует в поиске

```
car  = 63 61 72 00
cart = 63 61 72 74 00
```

- Байты записаны в шестнадцатеричном виде; `00` обозначает конец ключа.
- После `car` есть два перехода: конец `00` и буква `t` (74).
- Поэтому каждый ключ может заканчиваться отдельным листом.

Наш пример ASCII-ключей; demo/art_model.py: key_bytes.

ART для той же таблицы из четырёх строк



`prefix` — сжатый участок: сравниваем его байты без развилок.

После ребра с проверяем a; после d — o, g, `00`.

`00` — один байт конца ключа; в листьях — номер строки таблицы.

Проследим поиск cart до строки 42

Где мы	Проверяем prefix	Берём переход	Остаток
Корень	ϵ	c	art00
Узел после c	a	r	t00
Узел после car	ϵ	t	00
Лист	00	—	пусто $\Rightarrow$ 42

Для cap в узле после c не найдётся перехода p.

Полный поиск в исполняемой модели ART

```
38 def lookup(node, key):
39     remaining = key_bytes(key)
40     while node is not None:
41         if not remaining.startswith(node.prefix):
42             return None
43         remaining = remaining[len(node.prefix):]
44         if node.row_id is not None:
45             return node.row_id if not remaining else None
46         byte, remaining = remaining[0], remaining[1:]
47         node = node.children.get(byte)
48     return None
```

`row_id` — идентификатор строки таблицы: для `car` возвращаем 17.

`children.get(byte)` выбирает ребёнка. Именно этот шаг ART реализует четырьмя способами.

Учебная реализация: art_model.py

Node4: ищем байт в коротком массиве

Номер позиции	0	1	2	3
`key`	`r`	`t`	—	—
`children`	p_r	p_t	—	—

- Наш узел после с хранит `prefix=a`, `count=2`.
- Ищем `r`: находим позицию 0 и читаем `children[0]`.
- Проверяем не более четырёх однобайтовых меток.

Leis, Kemper, Neumann. ART, ICDE 2013; DuckDB node4.cpp.

Зачем Node16, если поиск такой же, как в Node4?

```
13 idx_t Node16::GetChildPos(uint8_t k) {
14   for (idx_t pos = 0; pos < count; pos++) {
15     if (key[pos] == k) {
16       return pos;
17     }
18   }
19   return Node::GetChildPos(k);
20 }
```

В DuckDB v0.5.0 обе формы ищут циклом по занятым позициям.

Node4: массивы на 4 метки и 4 ссылки. Node16: на 16 и 16.

Node4 экономит память у малых развилок; Node16 вмещает больше детей.

DuckDB v0.5.0, commit 109f932c41fb: node4.hpp, node16.hpp, node16.cpp, 13–20.

Node16 в libart сравнивает байты параллельно

```
170             #ifdef __amd64__
171                 // Compare the key to all 16 stored keys
172                 __m128i cmp;
173                 cmp = _mm_cmpeq_epi8(_mm_set1_epi8(c),
174                         _mm_loadu_si128((__m128i*)p.p2->keys));
175 
176                 // Use a mask to ignore children that don't exist
177                 mask = (1 << n->num_children) - 1;
178                 bitfield = _mm_movemask_epi8(cmp) & mask;
```

`cmpeq` сравнивает 16 байтов; `movemask` собирает результаты в битовую маску.

libart 301046804af1, art.c, 170–178.

Node48: сначала найдём номер позиции

Допустим, после ca уже 18 продолжений: a..p, r, t. Показаны три из них.

Байт	child_index[байт]	children[позиция]
a	0	p_a
r	16	p_r
t	17	p_t
z	EMPTY	нет перехода

Таблица: 256 однобайтовых номеров позиций. Массив: 48 адресов детей.

Вместо 256 адресов храним $256 + 48 \cdot 8 = 640$ байт переходов.

Leis, Kemper, Neumann. ART, ICDE 2013; DuckDB node48.cpp.

Node48: проверка наличия и две индексации

```
13 idx_t Node48::GetChildPos(uint8_t k) {
14   if (child_index[k] == Node::EMPTY_MARKER) {
15     return DConstants::INVALID_INDEX;
16   } else {
17     return k;
18   }
19 }
```

Здесь возвращается сам байт k. Затем ребёнка берут как `children[child_index[k]]`.

DuckDB v0.5.0, commit 109f932c41fb: node48.cpp, 13–19 и 44–47.

Node256: байт сразу служит индексом массива

- Храним `children[256]`.
- Для байта `r` = 114 берём `children[114]`.
- Много детей — мало пустых ячеек; дополнительная таблица уже не нужна.

Leis, Kemper, Neumann. ART, ICDE 2013; DuckDB node256.cpp.

Четыре формы наследуют общий Node

Форма	Вместимость	Как найти ребёнка
Node4	4	поиск в `key[4]`
Node16	16	поиск в `key[16]`
Node48	48	таблица байт $\rightarrow$ позиция
Node256	256	байт $\rightarrow$ адрес

В DuckDB v0.5.0 все четыре класса наследуют `Node`.

В нём общие поля: `count`, `type`, `prefix`.

Методы выбора ребёнка переопределены в каждом классе.

DuckDB v0.5.0, commit 109f932c41fb: node.hpp; node4/16/48/256.hpp.

Вставка в ART: три знакомых случая

- Ключ уже есть: для PRIMARY KEY отвергаем дубликат.
- Не совпал prefix узла или хвост листа: создаём развилку в месте различия.
- Не хватает перехода: добавляем ребёнка; если массив заполнен — увеличиваем узел.

DuckDB v0.5.0, commit 109f932c41fb: art.cpp, Insert, 287–352.

Пятый ребёнок превращает Node4 в Node16

```
77      auto new_node = new Node16();
78      new_node->count = 4;
79      new_node->prefix = move(node->prefix);
80      for (idx_t i = 0; i < 4; i++) {
81        new_node->key[i] = n->key[i];
82        new_node->children[i] = n->children[i];
83        n->children[i] = nullptr;
84      }
85      // Delete old node and replace it with new node
86      delete node;
87      node = new_node;
88      Node16::Insert(node, key_byte, new_child);
```

Создаём другой объект: Node16 вместо заполненного Node4.

Переносим prefix и четыре перехода, удаляем старый узел.

Общая ссылка Node* теперь указывает на новый объект.

DuckDB v0.5.0, commit 109f932c41fb: node4.cpp, 77–88.

Удаление может уменьшить узел

- Удаляем row_id; если лист опустел, убираем переход к нему.
- Развилку с одним ребёнком соединяем со следующим сжатым участком.
- Если детей стало мало, переносим их в узел меньшей вместимости.

DuckDB v0.5.0, commit 109f932c41fb: art.cpp, Erase; node16.cpp, Erase.

Точный поиск: добавим ART к сравнению

Реализация	Ключ есть, нс	Ключа нет, нс
Массив + бинарный поиск	96.1	90.7
Хеш-таблица	9.5	9.9
Trie (26 переходов)	17.1	19.4
Сжатый trie	89.1	83.1
PATRICIA	99.6	103.4
libart	25.8	24.7

Те же 50 000 слов; все строки получены в одном C++-прогоне.

Сжатый trie: `std::map` **внутри узла** хранит его детей.

libart — реализация ART; код всей СУБД DuckDB здесь не измеряем.

E1: GCC 13.3 -O2, AMD Zen 5, одно ядро.

Что посчитал процессор при успешном поиске

Реализация	Инструкций	Ветвлений	Промахов
Trie (26 переходов)	172.5	33.1	0.001
Сжатый trie	422.1	122.0	0.037
PATRICIA	606.0	120.3	0.007
libart	295.8	79.6	0.011

На один запрос; промахи — ошибки предсказания ветвлений.

Среднее 5 прогонов perf; построение индекса исключено.

E3: demo/perf-results.csv; сырые отчёты и разброс приложены.

Подсчёт по префиксу: массив или дерево?

Реализация	нс на подсчёт совпадений
Массив + бинарный поиск	245.2
Trie (26 переходов)	11180.9
Сжатый trie	1204.2
PATRICIA	603.2
libart	628.2

512 префиксов длиной до 3 символов; всего 63 796 совпадений.

Считаем весь ответ, без готового счётчика в узле.

Массив: `lower_bound` и последовательный обход совпадений.

E2: demo/results.csv; медиана 7 проходов.

Счётчики для подсчёта по префиксу

Реализация	Инструкций	Промахов ветвлений
Массив + бинарный поиск	4216.5	3.858
Trie (26 переходов)	117755.2	942.924
Сжатый trie	6072.6	39.951
PATRICIA	2948.5	3.182
libart	8060.4	13.915

На один запрос; среднее 5 прогонов perf. Ответы совпадают.

E3: demo/perf-results.csv.

Сколько байт занимает один узел ART?

Узел	Модель статьи, Б	sizeof libart, Б
Node4	52	56
Node16	160	160
Node48	656	656
Node256	2 064	2 064

Включены заголовок и переходы. Листья, ключи и расходы на выделение памяти сюда не входят.

Leis, Kemper, Neumann. ART, ICDE 2013, табл. I; sizeof из нашего C++ benchmark.

Что означает оценка 43 байта на ключ?

- В статье это верхняя оценка памяти внутренних узлов при 32-битных ключах.
- Для ключей произвольной длины в той же модели получается 52 Б/ключ.
- Листья, полные ключи и расходы выделения памяти в эту оценку не включены.

Leis, Kemper, Neumann. ART, ICDE 2013, табл. II.

Сохранить индекс — с двумя дополнительными условиями

- После перезапуска индекс должен работать без построения заново.
- Открытие базы не должно читать весь индекс: загружаем узлы по запросу.
- Ссылки на диск не должны увеличивать размер узлов в памяти.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022

Сериализация: что мы хотим восстановить?

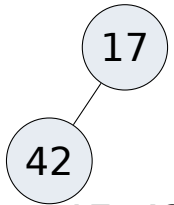
- Сериализация переводит данные и связи в сохраняемую запись.
- Десериализация восстанавливает их из этой записи.
- Для словаря можно сохранить только пары ключ–значение. Для той же формы дерева нужно записать ещё и связи.

Вариант 1: сохранить элементы, затем построить индекс

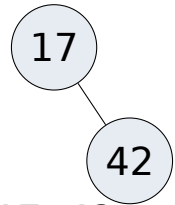
Ключ	Значение
car	17
cart	42
cat	8
dog	31

Читаем пары и заново выполняем вставки.
Содержимое восстановится; исходная форма в файл не попала.

Одних значений недостаточно для восстановления формы



17, 42



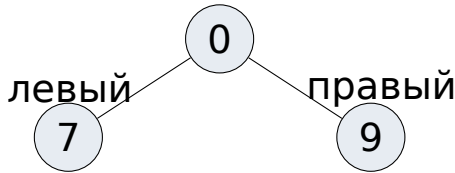
17, 42

Обход «узел, левое поддерево, правое поддерево» даёт одинаковые значения. Но ребёнок 42 находится с разных сторон.

Исключение: форма известна заранее

- У двоичной кучи уровни заполнены сверху вниз, последний — слева направо.
- В массиве с нумерацией от 1 дети узла v находятся в $2v$ и $2v + 1$.
- Для произвольного дерева этот способ оставляет дыры: правая цепочка занимает позиции 1, 3, 7, 15, …

Вариант 2: обход с маркерами отсутствующих детей




```
node(0), node(7), null, null,
node(9), null, null
```

Сначала узел, затем левое и правое поддеревья.
`null` — отдельный тег, поэтому значение 0 допустимо.

Сериализатор записывает каждый тег один раз

```
10 def encode(node):
11     tokens = []
12     def write(node):
13         if node is None:
14             tokens.append(('null',))
15         else:
16             tokens.append(('node', node.value))
17             write(node.left)
18             write(node.right)
19     write(node)
20     return tokens
```

$O(n)$ записей; дополнительный стек глубины $O(h)$.

Учебная реализация: serialization.py

Декодер восстанавливает одно поддерево

```
def decode(tokens, max_depth=100):
    stream = iter(tokens)
    def read(depth):
        if depth > max_depth:
            raise ValueError('depth limit')
        token = next(stream, None)
        if token == ('null',):
            return None
        if (not isinstance(token, tuple) or len(token) != 2
                or token[0] != 'node' or type(token[1]) is not int):
            raise ValueError('missing or invalid token')
        return Node(token[1], read(depth + 1), read(depth + 1))
```

`null` возвращает пустое дерево. `node` требует ещё два поддерева.

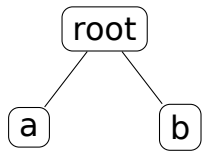
Учебная реализация: serialization.py

Форма бинарного дерева стоит $2n + 1$ бит

- У n узлов есть $2n$ мест для детей.
- $n - 1$ мест занято рёбрами; значит, пустых мест $n + 1$.
- Записываем n тегов node и $n + 1$ тегов null. Значения хранятся отдельно.

Если детей много: отмечаем начало и конец узла

У узла может быть любое число детей. Их список закрывает отдельный маркер.




```
BEGIN root
  BEGIN a
  END
  BEGIN b
  END
END
```

Devicetree Blob (.dtb) — бинарное описание оборудования: процессоров, памяти, устройств. При загрузке его передают ядру ОС.

Границы узлов: `FDT_BEGIN_NODE` / `FDT_END_NODE`.

Devicetree Specification, §5 и §5.4: Flattened Devicetree Format.

Вариант 3: пронумеруем узлы и сохраним ссылки

ID	Значение	Левый ID	Правый ID
0	0	1	2
1	7	null	null
2	9	null	null

Корень имеет ID 0. `null` отличается от любого ID.
Имена узлов не зависят от адресов процесса.

Восстановление по ID: сначала узлы, затем связи

```
28 def restore_table(rows, root_id):
29     # IDs are validated; links may be shared or cyclic.
30     nodes = [Node(value) for value, _, _ in rows]
31     for node, (_, left, right) in zip(nodes, rows):
32         node.left = None if left is None else nodes[left]
33         node.right = None if right is None else nodes[right]
34     return None if root_id is None else nodes[root_id]
```

Один ID обозначает один объект, даже если на него ссылаются несколько раз.

Учебная реализация: tree_records.py

Вариант 4: заменим ID адресами в файле

Смещение, Б	Значение	Левый, Б	Правый, Б
16	0	32	48
32	7	0	0
48	9	0	0

Заголовок — 16 Б; запись узла — 16 Б. Корень: смещение 16. Нулевое смещение означает отсутствие ребёнка.

По адресу можно прочитать один выбранный узел

```
71      def read(self, offset):
72          self.check(offset)
73          if offset == 0:
74              return None
75          self.stream.seek(offset)
76          value, left, right = RECORD.unpack(self.stream.read(RECORD.size))
77          self.check(left)
78          self.check(right)
79          self.loaded.append(offset)
80          return value, left, right
```

Прочитали корень по адресу 16, затем правого ребёнка по адресу 48.

Запись левого ребёнка по адресу 32 не нужна.

Учебная реализация: tree_records.py

Ссылка может хранить относительное смещение

- Вместо адреса процесса записываем расстояние до объекта.
- Например: поле находится в байте 24 и хранит 40; объект начинается в байте 64.
- FlatBuffers использует смещения для ссылок на таблицы, строки и векторы.

FlatBuffers, Internals: Format components / Offsets; flatbuffers.dev/internals/.

Теперь выберем формат для индекса DuckDB

Что сохраняем	Работа при открытии или запросе
Пары ключ–значение	Заново построить индекс
Обход с маркерами	Восстановить дерево; для прямых переходов добавить навигацию
Узлы и адреса детей	Прочитать корень, затем нужные узлы по пути

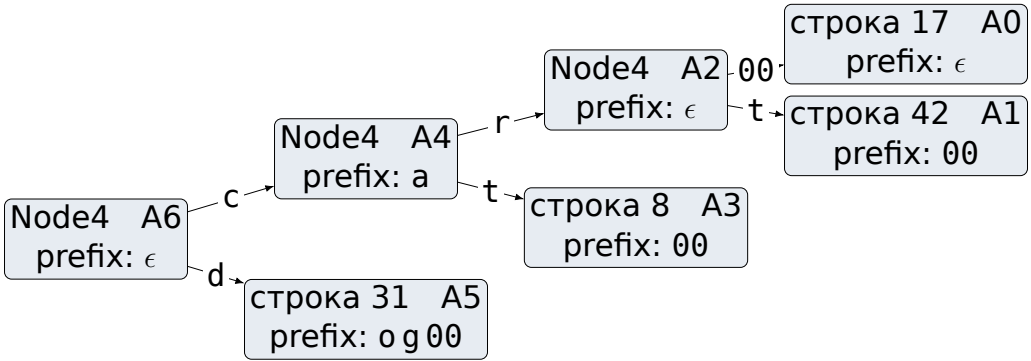
Требование: после открытия выполнить запрос `car`, не создавая всё дерево в памяти.
DuckDB хранит адреса детей; дальше разберём конкретный формат ART.

Адрес в файле: номер блока и смещение

- Файл состоит из блоков; узел начинается в определённом месте блока.
- Пара (`block_id, offset`) позволяет найти узел после перезапуска.
- Родитель хранит такие пары для детей; метаданные индекса — адрес корня.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022; DuckDB v0.5.0, commit 109f932c41fb

Запишем детей раньше родителя



Порядок записи: A0, A1, A2, A3, A4, A5, A6.

Когда пишем A2, адреса его детей A0 и A1 уже известны.

Какие связи окажутся в записи?

Адрес	Данные узла	Переходы
A0	лист; row_id = 17	—
A1	лист; row_id = 42	—
A2	Node4; prefix пуст	00→A0, t→A1
A4	Node4; prefix = a	r→A2, t→A3
A6	корень; prefix пуст	c→A4, d→A5

Показана часть записей. A0…A6 — адреса узлов, а row_id — номера строк таблицы.

DuckDB: рекурсивно сохраняем детей

```
62  vector<BlockPointer> child_offsets;
63  for (idx_t i = 0; i < internal_type.children_size; i++) {
64    child_offsets.emplace_back(internal_type.children[i].Serialize(art,
      writer));
65  }
66  auto block_id = writer.block->id;
67  uint32_t offset = writer.offset;
68  // Write Node Type
69  writer.Write(type);
70  // Write count
71  writer.Write(count);
72  // Write Prefix
73  prefix.Serialize(writer);
```

`child_offsets` собирает адреса уже записанных детей.

После этого фиксируем адрес текущего узла.

DuckDB v0.5.0, commit 109f932c41fb: node.cpp, 62–73.

DuckDB: записываем метки и адреса переходов

```
74    // Write Key values
75    for (idx_t i = 0; i < internal_type.key_size; i++) {
76      writer.Write(internal_type.key[i]);
77    }
78    // Write child offsets
79    for (auto &offsets : child_offsets) {
80      writer.Write(offsets.block_id);
81      writer.Write(offsets.offset);
82    }
83    return {block_id, offset};
84  }
```

В файле остаются байты переходов и пары `(block, offset)`.

DuckDB v0.5.0, commit 109f932c41fb: node.cpp, 74–84.

Ошибка в порядке чтения двух полей

До исправления: два чтения внутри одного вызова.

```
424     bound_info->indexes.emplace_back(reader.Read<idx_t>(),
        reader.Read<idx_t>());
```

C++ не задаёт порядок вычисления этих аргументов. Исправление:

```
424     auto idx_block_id = reader.Read<idx_t>();
425     auto idx_offset = reader.Read<idx_t>();
426     bound_info->indexes.emplace_back(idx_block_id, idx_offset);
```

Теперь сначала читаем номер блока, затем смещение.

DuckDB commit 6a8d01e, 16.06.2022; checkpoint_manager.cpp.

После открытия дети могут ещё лежать на диске

- Родитель уже загружен; его дети — не обязательно.
- Для каждой связи нужно знать: это адрес узла в памяти или адрес записи в файле?
- Хранить оба адреса одновременно дорого: в Node256 это ещё 256 пар.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022

Одна 64-битная ячейка хранит один из двух адресов

- Узел ещё на диске: в ячейке флаг, номер блока и смещение.
- Узел уже загружен: в той же ячейке указатель на него.
- При первом доступе читаем узел и заменяем дисковую ссылку указателем.

DuckDB v0.5.0, commit 109f932c41fb: SwizzleablePointer.

Как эта версия кодирует ссылку на диск



Старший бит 1: ссылка на диск. Нулевая ячейка: ребёнка нет.

После загрузки ячейка содержит обычный адрес узла.

DuckDB v0.5.0, commit 109f932c41fb: swizzleable_pointer.cpp, 19–26.

Упаковка номера блока и смещения

```
19  idx_t pointer_size = sizeof(pointer) * 8;
20  pointer = block_id;
21  pointer = pointer << (pointer_size / 2);
22  pointer += offset;
23  // Set the left most bit to indicate this is a swizzled pointer and send
      it back to the mother-ship
24  uint64_t mask = 1;
25  mask = mask << (pointer_size - 1);
26  pointer |= mask;
```

1 в старшем бите помечает дисковую ссылку.

DuckDB v0.5.0, commit 109f932c41fb: swizzleable_pointer.cpp, 19–26.

Первый доступ загружает ребёнка и меняет ссылку

```
64 Node *SwizzleablePointer::Unswizzle(ART &art) {
65   if (IsSwizzled()) {
66     // This means our pointer is not yet in memory, gotta deserialize this
67     // first we unset the bae
68     auto block_info = GetSwizzledBlockInfo();
69     *this = Node::Deserialize(art, block_info.block_id, block_info.offset);
70   }
71   return (Node *)pointer;
72 }
```

`Node::Deserialize` создаёт узел. Присваивание сохраняет указатель в том же месте.

DuckDB v0.5.0, commit 109f932c41fb: swizzleable_pointer.cpp, 64–72.

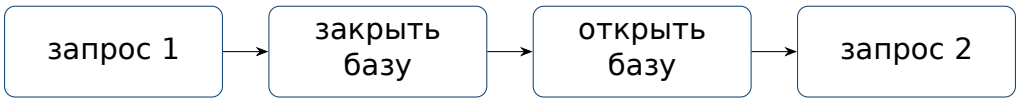
На ревью проверяют больше, чем алгоритм

- Кто освобождает загруженный узел? Повторяющийся код удаления предложили перенести в общий класс ссылки.
- Зачем новая пара чисел, если уже есть тип BlockPointer?
- Как прочитать файл, когда появится другой индекс? Ревьюер попросил сохранять тип индекса.

DuckDB PR #4022, ревью 4–5.07.2022.

Как проверить сохранение на уже готовых тестах?

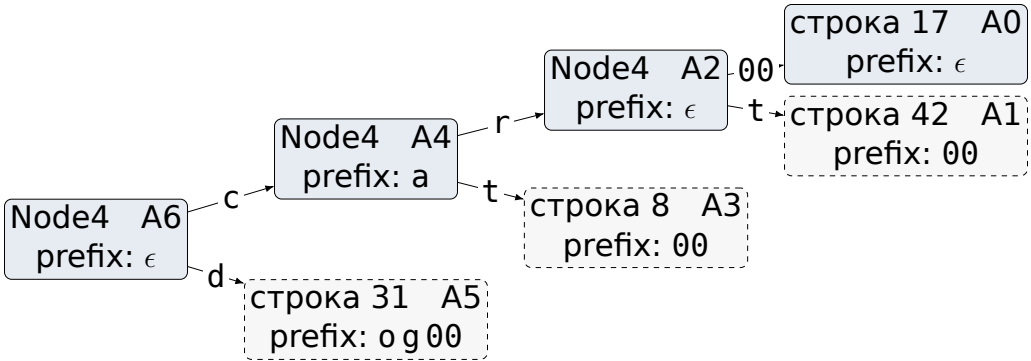
В PR #4022 расширили режим `--force-reload`:



- Между запросами теста принудительно переоткрываем базу.
- Те же ожидаемые ответы проверяют ещё и сохранение состояния.

DuckDB PR #4022; commit b11c87a; –force-reload.

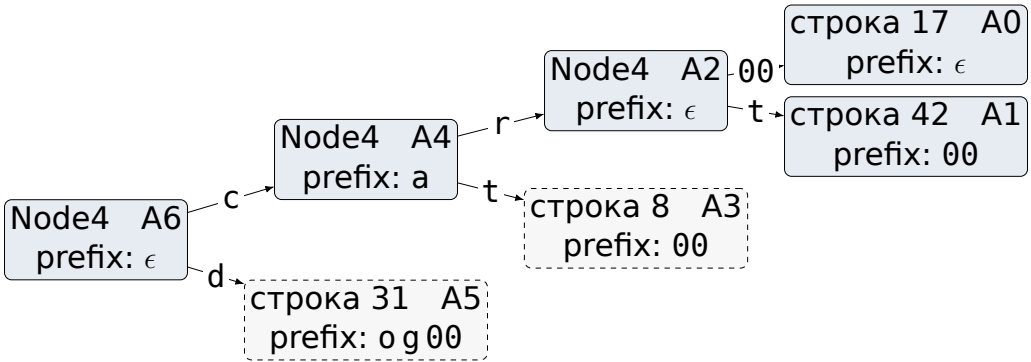
Первый запрос car: загружены четыре узла



Путь: A6 → A4 → A2 → A0. Ответ — строка 17.

Синие узлы загружены; пунктирные остаются в файле.

Следующий запрос cart: нужен ещё один узел



A6, A4 и A2 уже в памяти. Загружаем только A1 и получаем строку 42.

Быстрее открываем, дольше сохраняем

Операция	Без сохранения ART	С сохранением ART
Открытие базы, с	7,75	0,06
Закрытие с записью, с	8,99	18,97

50 млн целых ключей с PRIMARY KEY; замер авторов 2022 года.

Теперь при закрытии пишем и таблицу, и индекс. В листьях — ещё 50 млн 64-битных `row_id`: идентификаторов строк.

При открытии читаем метаданные; узлы загрузим по запросам.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022

Почему первые запросы стали дороже?

- В версии с сохранённым ART первые запросы загружают узлы по пути.
- В исходной версии эти узлы уже созданы при открытии базы.
- В опыте авторов холодная серия запросов была примерно втрое медленнее; после прогрева времена сблизились.

Pedro Holanda. Persistent Storage of ART in DuckDB, 27.07.2022, Query Time (Cold/Hot).

Что оставили на следующий этап?

- При сохранении состояния (checkpoint) первая версия переписывала весь индекс, даже если он не изменился.
- При чтении нескольких узлов из одного блока повторяли обращение к менеджеру памяти.
- Эти расходы оставили для следующей итерации: быстрый старт уже работал.

DuckDB PR #4022; Holanda, 27.07.2022, Future Work.

2023: PRIMARY KEY увеличил файл с 10 до 300 МБ

- В примере пользователя — 5 млн строк с временем и целым значением. **Весь файл базы** вырос в 30 раз после добавления индекса.
- Колонки хорошо сжимаются. ART хранится отдельно: развилки, части ключей и ссылки на строки.
- Убрать PRIMARY KEY? Пользователю нужна уникальность, чтобы повторная загрузка после сбоя не создавала дубликаты.

DuckDB issue #6667, 10–13.03.2023; данные пользователя.

Убрали отдельные листья. Где осталась память?

`row_id` — идентификатор строки таблицы. Если он один, храним его в ячейке ссылки, без отдельного листа.
Профиль **памяти** после изменения, август 2023:

Объект	Количество	Память, МБ
LEAF	0	0
PREFIX	4 939 552	≈120
NODE256	69 135	≈143

Основной расход остался: префиксы и массивы переходов Node256. Несовпадающие хвосты ключей не объединить.

Tania F., issue #6667, 24.08.2023; PR #8112.

Кому помог row_id прямо в ссылке?

100 000 значений	До, МБ	После, МБ
Короткие строки	6,0	4,2
Длинные строки	6,8	5,0
BIGINT: мало повторов	4,2	2,4
Много повторов INTEGER	2,1	2,1

Замеры авторов PR #8112, июль 2023.

Выигрыш есть, когда у ключа один row_id: отдельный лист больше не нужен.

DuckDB PR #8112, ART: inline row IDs into node pointers.

Какой индекс мы построили?

- Trie превратил ключ в путь; сжатие убрало промежуточные узлы.
- ART подбирает хранение переходов под число детей.
- Адреса в файле и заменяемые ссылки позволяют сохранить индекс и загружать его по запросу.

Спасибо за внимание